BUSINESS ACCOUNT MANAGEMENT SYSTEM

System Documentation, Database Schema & Relationship Diagrams

PROJECT PROFILE	SPECIFICATION
Project Type	Business Accounting Software
Backend	Core PHP (without Laravel)
Database	MySQL 8.0 / InnoDB
Frontend	HTML, CSS, JavaScript, Bootstrap
Database Access	MySQLi (without PDO)
Architecture	Modular, multi-company, multi-user
Accounting Method	Double-entry accounting

Prepared as an implementation-ready starting specification

Document Contents

- 1. Project Overview & Scope
- 2. System Modules
- 3. System Architecture
- 4. Database Design Principles
- 5. Complete Database Schema
- 6. Entity Relationship Diagram (ERD)
- 7. Accounting Workflows & Examples
- 8. Reports & Business Rules
- 9. Security Requirements
- 10. Technology & Implementation Roadmap

1. Project Overview & Scope

The Business Account Management System is a web-based application for recording and monitoring a company's financial activities. It centralizes accounts, income, expenses, sales, purchases, customer and supplier balances, and financial reporting. The system is intended for small and medium-sized businesses, including retail, wholesale, service, trading, software, and manufacturing companies.

Main objectives

- Maintain accurate financial records and account balances.
- Track revenue, expenses, profit or loss, cash and bank balances.
- Manage sales invoices, purchases, customer receipts and supplier payments.
- Generate Profit & Loss, Balance Sheet, Cash Flow and Trial Balance reports.
- Provide secure access for multiple users and isolate each company's data.

2. System Modules

No.	Module	Main responsibilities
01	Dashboard	Revenue, expenses, profit, cash/bank balances, recent transactions and charts.
02	Company Management	Company profile, logo, address, currency and financial-year settings.
03	User & Role Management	User accounts, roles, permissions, activation and access control.
04	Chart of Accounts	Account codes, hierarchy and Assets, Liabilities, Equity, Revenue and Expense categories.
05	Account Management	Cash/bank accounts, opening balances, transfers, account ledger and statements.
06	Income Management	Sales-related and other income records, categories, receipts and history.
07	Expense Management	Expense categories, recording, approvals, receipts and expense reports.
08	Sales Management	Sales invoices, invoice items, discounts, taxes, returns and customer payments.
09	Purchase Management	Purchase invoices, purchase items, supplier payments and returns.
10	Customer & Supplier	Profiles, contact information, transaction history and outstanding balances.
11	Financial Transactions	Receipts, payments, journal entries, transfers and ledger posting.
12	Financial Reports	Profit & Loss, Balance Sheet, Cash Flow, Trial Balance and ledgers.

No.	Module	Main responsibilities
13	Employee & Payroll	Employee records, salary structure, payroll, bonuses and deductions.
14	Product & Inventory	Products, stock movements, stock alerts and inventory valuation.
15	Tax Management	Tax configuration, calculations, records and reports.
16	Audit Logs	Login history and records of financial and administrative changes.

Recommended first release

Build modules 01–12 first. Add payroll, inventory, tax and advanced audit functions after the central accounting workflow has been tested.

3. System Architecture

The application is divided into functional modules. Business modules create source records; the accounting engine converts approved business events into balanced journal entries; reports read posted ledger data.

Layer	Components
Presentation	HTML, CSS, Bootstrap, JavaScript; responsive pages and forms.
Application	Core PHP modules, validation, authentication, role checks and business rules.
Accounting Engine	Journal entry posting, debit/credit validation, ledger calculation and reversals.
Data	MySQL 8.0 relational tables using InnoDB, foreign keys and indexes.
Security	Password hashing, sessions, CSRF protection, prepared statements and company isolation.

High-level data flow

Sales / Purchases / Income / Expenses / Payroll / Transfers → Journal Entries → Journal Lines → Chart of Accounts → Account Ledger → Financial Reports.

4. Database Design Principles

- Each company owns its users and business records. Include company_id on company-owned records.
- Use InnoDB and foreign keys for relational integrity.
- Use DECIMAL(15,2) for money, not FLOAT or DOUBLE.
- Use journal_entries and journal_lines as the accounting source of truth.
- Only posted journal entries affect financial reports and account balances.
- Use database transactions when saving a source record and its journal entries together.
- Do not permanently delete posted financial records; use reversal entries.
- Enforce company ownership in application queries and, where practical, composite foreign keys.
- Use unique constraints such as (company_id, account_code) and (company_id, invoice_number).

5. Complete Database Schema

The following schema is a logical field-level design. INT primary keys should normally be AUTO_INCREMENT. Foreign-key columns should use the same type and signedness as their referenced primary keys. Add created_at and updated_at timestamps to operational tables where appropriate.

5.1 Company & Access Control

Table	Field	Type	Key / Notes
companies	company_id	INT	PK, AUTO_INCREMENT
	company_name	VARCHAR(150)	NOT NULL
	email	VARCHAR(150)	NULL
	phone	VARCHAR(30)	NULL
	address	TEXT	NULL
	logo	VARCHAR(255)	NULL
	currency	VARCHAR(10)	Default BDT
	financial_year_start	DATE	NULL
	status	TINYINT	Default 1
users	user_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	name	VARCHAR(120)	NOT NULL
	email	VARCHAR(150)	NOT NULL
	password	VARCHAR(255)	Password hash
	status	TINYINT	Default 1
roles	role_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	role_name	VARCHAR(80)	Unique per company
user_roles	user_role_id	INT	PK, AUTO_INCREMENT
	user_id	INT	FK → users
	role_id	INT	FK → roles

5.2 Customers & Suppliers

Table	Field	Type	Key / Notes
customers	customer_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	customer_name	VARCHAR(150)	NOT NULL
	phone	VARCHAR(30)	NULL
	email	VARCHAR(150)	NULL
	address	TEXT	NULL
	status	TINYINT	Default 1
suppliers	supplier_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	supplier_name	VARCHAR(150)	NOT NULL
	phone	VARCHAR(30)	NULL
	email	VARCHAR(150)	NULL
	address	TEXT	NULL
	status	TINYINT	Default 1

5.3 Chart of Accounts & Accounting Engine

Table	Field	Type	Key / Notes
account_categories	category_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	category_name	VARCHAR(100)	NOT NULL
	account_type	ENUM	ASSET, LIABILITY, EQUITY, REVENUE, EXPENSE
	description	TEXT	NULL
accounts	account_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	category_id	INT	FK → account_categories
	parent_account_id	INT	Self FK → accounts, NULL allowed
	account_code	VARCHAR(30)	Unique per company
	account_name	VARCHAR(120)	NOT NULL
	opening_balance	DECIMAL(15,2)	Default 0
	opening_balance_side	ENUM	DEBIT / CREDIT
	is_active	TINYINT	Default 1
journal_entries	journal_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	journal_number	VARCHAR(50)	Unique per company
	transaction_date	DATE	NOT NULL
	reference_number	VARCHAR(100)	NULL
	description	TEXT	NULL
	source_type	VARCHAR(50)	e.g. SALE, EXPENSE, PAYMENT
	source_id	INT	Source record ID; app-validated

Table	Field	Type	Key / Notes
	status	ENUM	DRAFT, POSTED, REVERSED
	created_by	INT	FK → users
journal_lines	line_id	INT	PK, AUTO_INCREMENT
	journal_id	INT	FK → journal_entries
	account_id	INT	FK → accounts
	description	TEXT	NULL
	debit	DECIMAL(15,2)	Default 0
	credit	DECIMAL(15,2)	Default 0

5.4 Income & Expense

Table	Field	Type	Key / Notes
income_categories	income_category_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	category_name	VARCHAR(100)	NOT NULL
	account_id	INT	FK → accounts (revenue)
incomes	income_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	income_category_id	INT	FK → income_categories
	account_id	INT	FK → accounts (cash/bank)
	amount	DECIMAL(15,2)	Must be positive
	income_date	DATE	NOT NULL
	description	TEXT	NULL
	journal_id	INT	FK → journal_entries
	created_by	INT	FK → users
expense_categories	expense_category_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	category_name	VARCHAR(100)	NOT NULL
	account_id	INT	FK → accounts (expense)
expenses	expense_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	expense_category_id	INT	FK → expense_categories
	account_id	INT	FK → accounts (cash/bank)
	amount	DECIMAL(15,2)	Must be positive
	expense_date	DATE	NOT NULL
	description	TEXT	NULL
	receipt_file	VARCHAR(255)	NULL
	journal_id	INT	FK → journal_entries
	created_by	INT	FK → users

5.5 Sales & Purchases

Table	Field	Type	Key / Notes
sales	sale_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	customer_id	INT	FK → customers
	invoice_number	VARCHAR(50)	Unique per company
	sale_date	DATE	NOT NULL
	subtotal	DECIMAL(15,2)	Default 0
	discount	DECIMAL(15,2)	Default 0
	tax_amount	DECIMAL(15,2)	Default 0
	total_amount	DECIMAL(15,2)	Default 0
	status	ENUM	DRAFT, POSTED, PARTIAL, PAID, CANCELLED
	journal_id	INT	FK → journal_entries, NULL until posted
sales_items	sales_item_id	INT	PK, AUTO_INCREMENT
	sale_id	INT	FK → sales
	product_id	INT	FK → products
	quantity	DECIMAL(12,2)	Must be positive
	unit_price	DECIMAL(15,2)	NOT NULL
	total	DECIMAL(15,2)	NOT NULL
purchases	purchase_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	supplier_id	INT	FK → suppliers
	invoice_number	VARCHAR(50)	Unique per company
	purchase_date	DATE	NOT NULL
	subtotal	DECIMAL(15,2)	Default 0
	discount	DECIMAL(15,2)	Default 0
	tax_amount	DECIMAL(15,2)	Default 0
	total_amount	DECIMAL(15,2)	Default 0
	status	ENUM	DRAFT, POSTED, PARTIAL, PAID, CANCELLED
	journal_id	INT	FK → journal_entries, NULL until posted
purchase_items	purchase_item_id	INT	PK, AUTO_INCREMENT
	purchase_id	INT	FK → purchases
	product_id	INT	FK → products
	quantity	DECIMAL(12,2)	Must be positive
	unit_cost	DECIMAL(15,2)	NOT NULL
	total	DECIMAL(15,2)	NOT NULL

5.6 Payments

Table	Field	Type	Key / Notes
payments	payment_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	party_type	ENUM	CUSTOMER / SUPPLIER / OTHER
	customer_id	INT	FK → customers, nullable
	supplier_id	INT	FK → suppliers, nullable
	account_id	INT	FK → accounts (cash/bank)
	payment_type	ENUM	RECEIPT / PAYMENT
	amount	DECIMAL(15,2)	Must be positive
	payment_date	DATE	NOT NULL
	payment_method	VARCHAR(50)	Cash, bank, card, etc.
	reference_number	VARCHAR(100)	NULL
	journal_id	INT	FK → journal_entries
	created_by	INT	FK → users
payment_allocations	allocation_id	INT	PK, AUTO_INCREMENT
	payment_id	INT	FK → payments
	invoice_type	ENUM	SALE / PURCHASE
	invoice_id	INT	Invoice ID; validate type and company
	allocated_amount	DECIMAL(15,2)	Must be positive

5.7 Products & Inventory

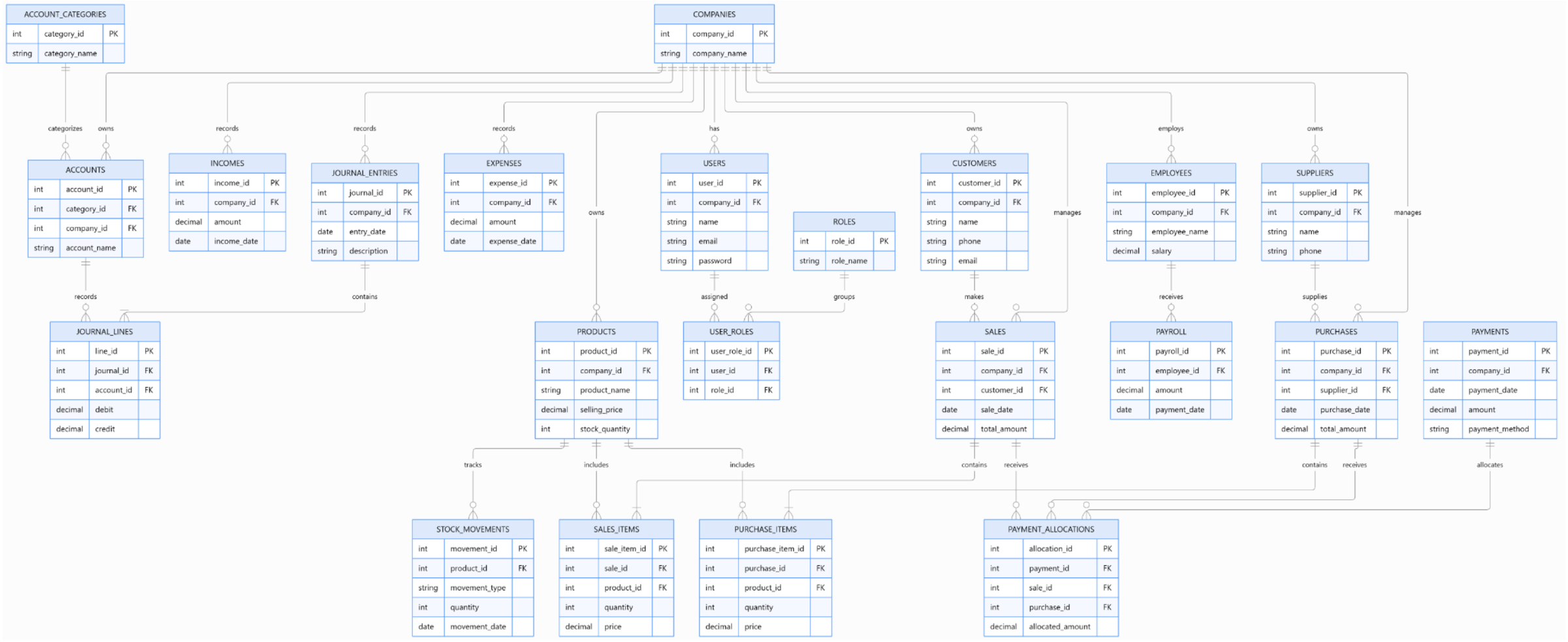
Table	Field	Type	Key / Notes
products	product_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	product_name	VARCHAR(150)	NOT NULL
	product_code	VARCHAR(50)	Unique per company
	purchase_price	DECIMAL(15,2)	Default 0
	selling_price	DECIMAL(15,2)	Default 0
	current_stock	DECIMAL(12,2)	Summary/cache value
	status	TINYINT	Default 1
stock_movements	movement_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	product_id	INT	FK → products
	movement_type	ENUM	IN / OUT / ADJUSTMENT
	quantity	DECIMAL(12,2)	NOT NULL
	reference_type	VARCHAR(50)	Purchase, sale, adjustment
	reference_id	INT	Source record ID; app-validated
	movement_date	DATE	NOT NULL

5.8 Employees, Payroll & Audit

Table	Field	Type	Key / Notes
employees	employee_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	employee_name	VARCHAR(150)	NOT NULL
	phone	VARCHAR(30)	NULL
	designation	VARCHAR(100)	NULL
	joining_date	DATE	NULL
	basic_salary	DECIMAL(15,2)	Default 0
	status	TINYINT	Default 1
payroll	payroll_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	employee_id	INT	FK → employees
	salary_month	DATE	Use first day of month
	basic_salary	DECIMAL(15,2)	NOT NULL
	bonus	DECIMAL(15,2)	Default 0
	deductions	DECIMAL(15,2)	Default 0
	net_salary	DECIMAL(15,2)	Calculated amount
	payment_status	ENUM	PENDING / PAID
	journal_id	INT	FK → journal_entries
audit_logs	log_id	INT	PK, AUTO_INCREMENT
	company_id	INT	FK → companies
	user_id	INT	FK → users
	action	VARCHAR(100)	NOT NULL
	table_name	VARCHAR(100)	NOT NULL
	record_id	INT	NULL
	description	TEXT	NULL
	created_at	TIMESTAMP	Default current timestamp

6. Entity Relationship Diagram (ERD) — Full Database

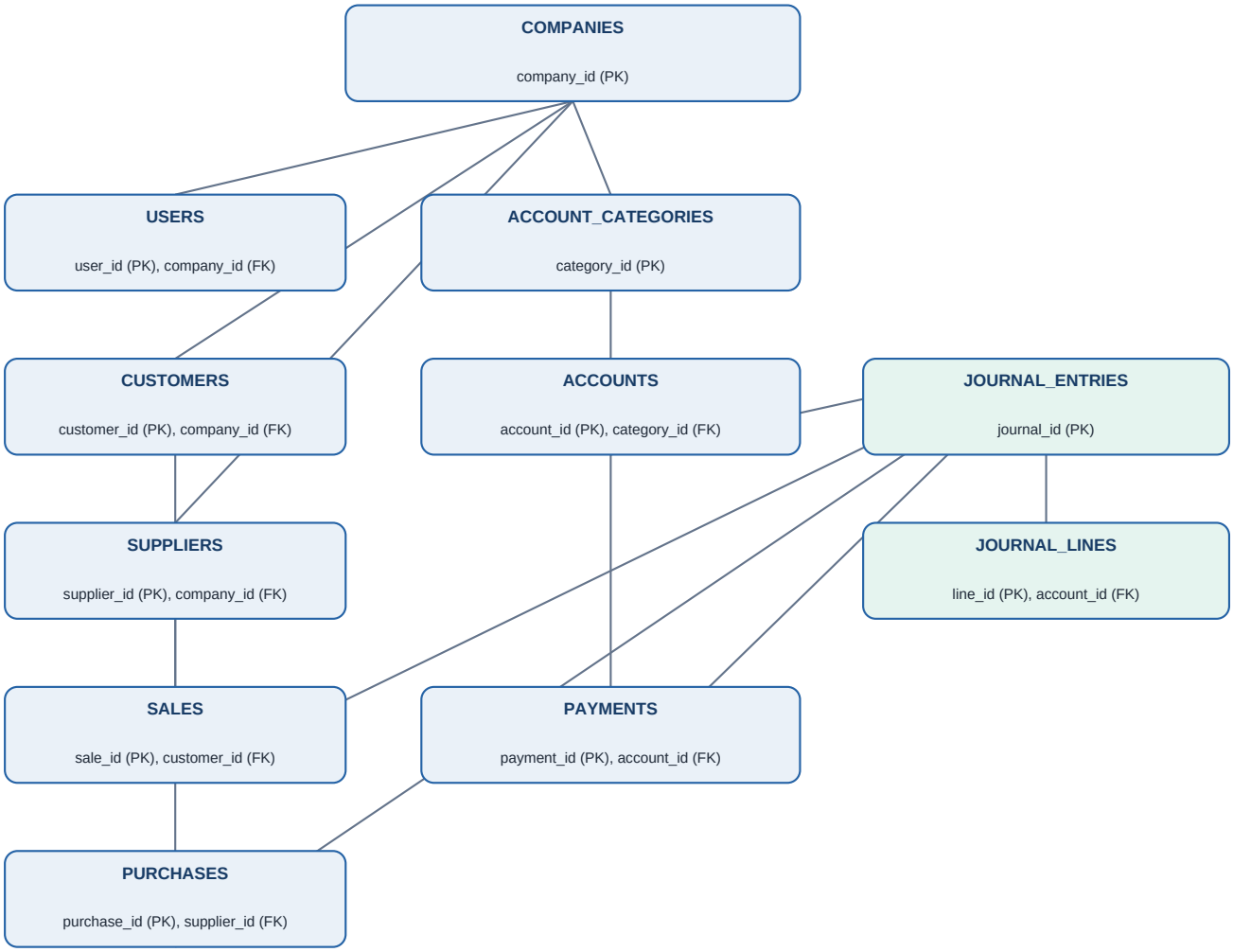
Business Account Management System



Full relationship diagram supplied for the database design

6. Entity Relationship Diagram (ERD)

The ERD below shows the principal relationships. The full database also includes supporting relationships for categories, payments, payroll, products and audit records.



Simplified ERD: supporting tables such as sales_items, purchase_items, payroll and stock_movements are described in the schema.

Key relationships

- One company has many users, roles, customers, suppliers, accounts and journal entries.
- One account category contains many accounts; an account can optionally have a parent account.
- One journal entry contains two or more journal lines; each line references one account.
- One customer can have many sales invoices; one supplier can have many purchase invoices.
- Payments are allocated to one or more invoices through payment_allocations.
- Sales and purchases can create stock movements and journal entries when posted.

7. Accounting Workflows & Examples

The system uses double-entry accounting. Every posted journal must satisfy total debits = total credits. The journal is created in the same database transaction as the related business record so that partial posting cannot occur.

Example A — Customer pays an outstanding invoice (■10,000)

Account	Debit	Credit
Cash / Bank	■10,000	—
Accounts Receivable	—	■10,000
Total	■10,000	■10,000

Example B — Company pays office rent in cash (■5,000)

Account	Debit	Credit
Rent Expense	■5,000	—
Cash in Hand	—	■5,000
Total	■5,000	■5,000

Profit calculation

Net Profit = Total Revenue – **Total Expenses**
Illustration: Revenue ■850,000 – Expenses ■250,000 = Net Profit ■600,000. For accrual accounting, revenue and expenses are recognized when earned or incurred, not only when cash moves.

Posting rules

- Draft transactions do not affect account balances or financial statements.
- Posting validates the journal, account ownership, date, and debit/credit totals.
- Posted transactions are immutable; corrections use a linked reversal and replacement entry.
- A transfer between company bank and cash accounts is not income or expense.
- A customer receipt against an existing invoice reduces receivables; it is not new revenue.
- Inventory purchases and cost of goods sold need inventory accounting rules when inventory is enabled.

8. Reports & Business Rules

Report	Calculation / Purpose
Profit & Loss	Revenue less expenses for a selected period; use posted ledger entries.
Balance Sheet	Assets = Liabilities + Equity as of a selected date.
Cash Flow	Cash and cash-equivalent inflows and outflows by activity.
Trial Balance	Lists account debit and credit balances; totals must agree.
Account Ledger	Chronological posted journal lines for one account.
Customer Ledger	Invoices, receipts, credit notes and customer balance.
Supplier Ledger	Purchases, payments, debit notes and supplier balance.
Sales / Purchase / Expense	Operational analysis by date, party, category and status.

9. Security Requirements

- Hash passwords using PHP password_hash() and verify with password_verify().

- Use MySQLi prepared statements for every user-supplied database value.
- Regenerate session IDs after login; enforce session timeout and logout.
- Check permissions on the server for every protected action.
- Validate company_id from the authenticated session; never trust a submitted company_id.
- Use CSRF tokens for state-changing forms and escape output to prevent XSS.
- Validate uploaded file type, size and storage location.
- Keep backups and test database restoration.
- Log sensitive changes; restrict access to audit logs.

10. Technology & Implementation Roadmap

Technology	Use
Core PHP	Backend modules and business logic; no Laravel.
MySQL 8.0	Relational database with InnoDB and foreign keys.
MySQLi	Database access using prepared statements; no PDO.
HTML / CSS / Bootstrap	Responsive user interface.
JavaScript	Client-side interactions and form assistance.
Apache / XAMPP	Local development and hosting environment.

Phase	Implementation tasks
1	Project setup, database, authentication and company isolation.
2	Roles, permissions, Chart of Accounts and Account Management.
3	Income, expenses, journal engine and account ledger.
4	Customers, suppliers, sales, purchases and payments.
5	Dashboard, Profit & Loss, Balance Sheet, Cash Flow and Trial Balance.
6	Inventory, payroll, tax, audit logs, testing and deployment.

Final Engineering Notes

This document is a logical design specification, not a complete executable SQL migration. Before production, define all foreign-key constraints, unique indexes, CHECK constraints, invoice allocation rules, journal posting transactions, and company-scoped access checks. In particular, polymorphic references such as source_type/source_id and invoice_type/invoice_id require application-level validation or a more normalized reference design.

END OF DOCUMENT